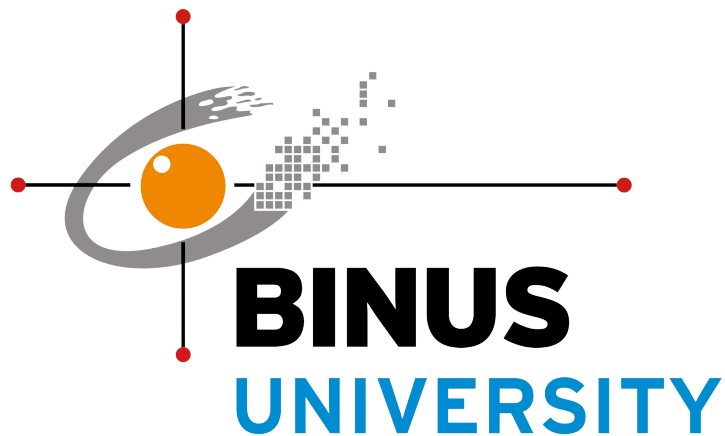


# **LAPORAN PROJECT ASSURANCE OF LEARNING**



## **JUDUL**

**Quotify: Quotes Selector Development**

Oleh:

Michael Dimas Chrispradipta - 2602132154

Yithro Paulus Tjendra - 2602112051

Machine Learning

Kelas: LB01

Tahun Ajaran: 2023/2024

# DAFTAR ISI

|                                                    |          |
|----------------------------------------------------|----------|
| <b>DAFTAR ISI.....</b>                             | <b>1</b> |
| <b>BAB 1. PENDAHULUAN.....</b>                     | <b>2</b> |
| 1.1. Latar Belakang.....                           | 2        |
| 1.2. Rumusan Masalah.....                          | 5        |
| 1.3. Tujuan dan Manfaat.....                       | 5        |
| 1.4. Luaran Kegiatan.....                          | 6        |
| <b>BAB 2. TINJAUAN PUSTAKA.....</b>                | <b>6</b> |
| 2.1. Kesehatan Mental dan Proses Belajar.....      | 6        |
| 2.2. Natural Language Processing.....              | 6        |
| 2.3. Semantic Analysis & 3 Model ML Percobaan..... | 7        |
| 2.4. Sistem Rekomendasi.....                       | 8        |
| 2.5. React JS.....                                 | 8        |
| 2.6. Flask.....                                    | 9        |
| <b>BAB 3. TAHAPAN PELAKSANAAN.....</b>             | <b>9</b> |
| 3.1. EDA dan Preprocessing Data.....               | 9        |
| 3.2. Perancangan Desain Model dan Aplikasi.....    | 15       |
| 3.3. Training-Testing Model.....                   | 15       |
| 3.4. Evaluasi Perangkat dan Uji Coba.....          | 24       |
| 3.5. Perancangan Aplikasi.....                     | 26       |
| 3.5.1. Desain Aplikasi.....                        | 26       |
| 3.5.2. Realisasi Aplikasi.....                     | 28       |
| 3.5.2.1. Front End.....                            | 29       |
| 3.5.2.2. Back End.....                             | 31       |

# BAB 1. PENDAHULUAN

## Abstrak:

Sepanjang sejarah, kehidupan manusia selalu dipenuhi gejolak emosional seperti duka, tekanan, kemarahan, bangga, dan kebahagiaan yang mendorong individu untuk mencari respons melalui berbagai cara, termasuk mencari kata-kata inspiratif dari tokoh-tokoh besar atau seseorang yang sangat berarti. ‘Tradisi mencari quotes’ telah ada sejak lama, mulai dari peradaban Yunani Kuno hingga era digital saat ini, dimana masyarakat menemukan makna dalam kutipan dari karya sastra, orator, film, hingga idola mereka. Kata-kata yang relevan dengan gejolak emosional seseorang dapat memberikan sudut pandang baru dan menjadi titik awal perubahan besar dalam hidupnya. Inilah yang mendasari pengembangan sistem **Quotify** sebagai *Quotes Selector* dalam proyek ini, yang bertujuan merekomendasikan quotes berdasarkan perasaan *user* untuk membantu menemukan pencerahan dan inspirasi yang berdampak pada tindakan mereka selanjutnya. Pada paper ini juga akan dijabarkan secara terperinci bagaimana sistem Quotify dibuat mulai dari pelatihan model hingga dalam bentuk utuh aplikasi, termasuk uraian pengujian 3 jenis model AI yang diujikan untuk mengklasifikasikan emosi berdasarkan suatu statement.. Proyek ini diharapkan dapat memfasilitasi ekspresi konkret dari quotes yang bermakna bagi *user*, mulai dari seni hingga motto hidup.

## 1.1. Latar Belakang

1. Cara alternatif untuk membantu mereka yang mengalami ‘kelelahan hidup’ dengan berbagai problematika atau gejolak emosi yang dirasakan
2. Memberi suatu sudut pandang baru dalam memberi makna atau melihat suatu hal
3. Proses belajar dari pengalaman atau pemaknaan tokoh-tokoh besar

Sepanjang sejarah umat manusia, kehidupan setiap individu kerap diwarnai oleh berbagai macam gejolak emosional. Gejolak tersebut direpresentasikan mulai dari duka mendalam karena kehilangan sosok yang berharga, merasa tertekan akibat bertumpuk-tumpuknya persoalan, kemarahan yang terpicu pengalaman ketidakadilan, rasa bangga saat mencapai kemenangan hingga rasa berbunga-bunga saat sukses dalam momentum romansa. Dalam kondisi demikian, manusia cenderung mencari suatu

‘tindak reaksi’ atas momentum gejolak yang dirasakannya tersebut. Mulai dari mencari teman untuk bercerita, merenungi pengalaman dalam kesunyian, berjumpa expertise untuk konsultasi, mengajak kenalan terdekat untuk merayakan, ataupun memposting hal-hal estetik-sentimental di akun media sosial. Termasuk juga dengan mencari quotes atau kata-kata mutiara atau kata-kata motivasional dari tokoh-tokoh besar mulai dari untuk menyemangati keadaan, mencari tuntunan, hingga menemukan ‘pencerahan’ dalam pengalaman seseorang.

‘Trend mencari quotes’ sejatinya telah berlangsung sejak sangat lama. Merupakan hal yang umum bagi masyarakat masa lampau, terlebih dalam peradaban Yunani Kuno, untuk menggantungkan tindak hidupnya pada kata-kata pencerahan dari para filsuf yang disebarkan dari mulut ke mulut. Lebih jauh lagi, kutipan-kutipan teks-teks religius yang diimani sebagai sabda dari Dia yang transendental ataupun tokoh-tokoh yang dipercaya sebagai ‘tangan kanan-Nya’ juga umum dianggap sebagai pegangan utama dalam menjalani kehidupan. Seiring berjalannya waktu, juga dengan berkembangnya proses jaringan informasi dalam globalisasi, masyarakat juga mulai mengenal dan menemukan kata-kata mutiara yang dirasa penuh makna terhadap gejolak-gejolak emosional tertentu, mulai dari kutipan dalam suatu karya sastra, pesan-pesan motivasional dari orator-orator terkemuka, potongan dialog dalam suatu film atau drama, hingga kesan-kesan hidup dari idola atau tokoh-tokoh yang telah sukses dalam suatu bidang tertentu.

Belajar dari pengalaman seseorang (*baik itu tokoh nyata ataupun fiksi*) menjadi poin terbesar momentum pencarian quotes. Kata-kata penuh makna, terlebih jika sangat ‘*relate*’ dengan gejolak emosional yang sedang dirasakan dapat memberi peran besar bagi seseorang untuk menemukan sudut pandang baru ataupun penegasan yang kemudian berkemungkinan sangat menentukan langkah besar yang diambil setelahnya, atau dalam kasus tertentu hingga menjadi titik awal momentum ‘turning point’ dalam kisah hidupnya. Seseorang mungkin menemukan tuntunan untuk keluar dari keputusasaan yang dialaminya, seseorang mungkin mengubah tabiatnya kala mendengar suatu kalimat inspirasional seorang tokoh yang merefleksikan hidupnya, seseorang mungkin juga dapat bersikap lebih bijaksana atas kegembiraan yang

dialaminya kala mengingat ucapan seseorang yang di masa lalunya pernah merasakan hal serupa. Poin-poin inilah yang kemudian mendorong kelompok kami untuk mengembangkan sistem fitur Quotes Selector dalam proyek ini.

## **1.2. Rumusan Masalah**

- a. Bagaimana cara agar model dapat memahami label perasaan manusia dari suatu percakapan?
- b. Bagaimana cara agar model dapat secara tepat memilih quote yang cocok dengan apa yang dirasakan user saat itu?
- c. Bagaimana teknik/metode deployment yang tepat agar fitur ini dapat diaplikasikan dengan optimal?

## **1.3. Tujuan dan Manfaat**

Didasarkan dengan melihat peran besar quotes dalam proses hidup seseorang seperti telah dijabarkan dalam 1.1, sistem Quotes Selector dalam proyek kali ini ditujukan untuk menawarkan suatu quotes dari seorang tokoh publik atau karakter fiktif berbasis pada statement gejala emosional yang dirasakan oleh user. Diharapkan fitur ini dapat membantu user menemukan suatu sudut pandang ataupun pencerahan dari kalimat suatu tokoh tertentu atas apa yang tengah dirasakannya yang kemudian berkemungkinan memberi peran sedikit-banyak dalam tindakan yang hendak diambil kedepannya. Lebih jauh lagi, manfaat dalam pengaplikasian ‘proses menemukan quotes’ ini dapat tercermin dalam bagaimana quotes yang dirasa sangat bermakna bagi user diekspresikan secara konkret. Contoh-contoh nyata dari hal ini mulai dari menuliskan quotes yang dirasakannya berharga itu di tempat yang dianggapnya spesial, membuat suatu karya seni atas quotes tersebut, menjadi motto hidup yang kemudian dituliskan dalam status atau profil Bio media sosialnya, menjadi tambahan dalam buku koleksi kumpulan quotes berharga yang dijumpai sepanjang hidupnya, serta berbagai macam bentuk ekspresi lainnya.

## **1.4. Luaran Kegiatan**

- Video Demonstrasi Aplikasi
- Laporan Akhir
- Prototipe Figma
- Produk Fungsional

# **BAB 2. TINJAUAN PUSTAKA**

## **2.1. Kesehatan Mental dan Proses Belajar**

Melihat cakupan luas dari apa yang menjadi inti dari proyek ini terhadap user, poin kesehatan mental tidak dapat dikesampingkan. Di samping berbagai macam isu mengenai kesehatan mental yang melanda (terlebih kalangan kaum) belakangan ini, peran menemukan quotes atau kata-kata mutiara yang secara tepat berkaitan dengan apa yang tengah dihadapi seseorang sedikit banyak akan berpengaruh pada kondisi mentalnya. Dalam kadar-kadar tertentu, mereka yang benar-benar menemukan suatu quotes yang tepat bermakna akan mengalami suatu transisi mental tersendiri sehingga tindakan atau sikapnya dapat berubah ataupun semakin menguat. Hal tersebut bisa jadi berupa perasaan tercerahkan atau bahkan perasaan terpukul kala mendengar suatu statement tertentu secara tidak sengaja berkonteks sangat mirip dengan apa yang dialaminya. Semua tetap bergantung bagaimana pribadi tersebut memaknainya, kesamaan kondisi yang ada, hingga pada siapa yang mengatakan statement tersebut. Secara personal, hal ini juga besar ditentukan oleh bagaimana pribadi tersebut mau terbuka, berpikir positif dan belajar dari refleksi makna seseorang atas suatu kejadian.

## **2.2. Natural Language Processing**

Melihat bahwa sistem yang dihasilkan dalam proyek ini bergantung besar besar proses training model semantic analysis dan sistem rekomendasi lewat pencocokan kemiripan kata, NLP dapat dikatakan mengambil peran besar dalam menentukan kualitas hasil. Seperti namanya, NLP merupakan suatu cabang dari ilmu AI yang berfokus pada bagaimana sistem komputer dapat memproses dengan bahasa yang umum dipakai oleh manusia sehingga dapat berinteraksi seperti melalui proses memahami,

menginterpretasikan hingga menghasilkan susunan kata seperti apa yang diucapkan manusia. Secara lebih jauh, melalui proses yang lebih dalam dan kompleks, NLP dapat diaplikasikan hingga taraf saran pengambilan keputusan, teman bicara yang suportif dalam berbagai macam bidang, hingga mengerjakan suatu task kompleks hanya berdasar pada apa yang diucapkan oleh user.

### **2.3. Semantic Analysis & 3 Model ML Percobaan**

Semantic Analysis merupakan salah satu proses dalam NLP di mana sistem berusaha untuk memahami teks didasarkan pada relasi kecocokan antar kata kunci dan struktur. Output dari bentuk pemahaman teks ini, salah satunya, dapat berupa pengklasifikasian suatu teks terhadap suatu label tertentu di antara sekian banyak teks yang ada, yang dalam hal ini adalah dengan memberi label ‘emotion’ pada suatu kalimat. Dalam proyek kali ini, terdapat tiga jenis model classifier yang diuji coba untuk dilatih pada dataset yang serupa untuk mencari model mana yang paling optimal dalam menentukan label emosi atas suatu kalimat, sehingga kemudian dapat diaplikasikan untuk mendukung proses rekomendasi pemilihan quotes. Ketiga model classifier tersebut adalah: Naive Bayes Classification, Logistic Regression, dan BERT.

Secara umum proses klasifikasi dalam Naive Bayes didasarkan pada Bayes Theorem pada asumsi hubungan independensi antar fitur dengan output berbasis pada kalkulasi probabilitasnya. Di samping itu, algoritma klasifikasi yang diterapkan dalam Logistic Regression lebih menitikberatkan pada pengambilan nilai maksimal dari probabilitas kalkulasi logistic function (sigmoid function) dari suatu input dengan serangkaian bobot fiturnya terhadap suatu kelas tertentu dalam menentukan label. Di sisi lain, BERT merupakan suatu pre-trained model yang dikembangkan oleh Google dengan berbasis pada transformer yang secara khusus dibuat untuk ranah NLP. Di samping serangkaian layer encoding Transformer nya yang membuatnya mampu menghasilkan word embedding secara bidirectional, model ini didasar pada proses *self attention mechanism* yang mampu memberi bobot secara khusus pada suatu kata dalam konteks keseluruhan teks. Secara praktikal terkait proyek kali ini, hasil performa dari ketiga model akan dijabarkan dalam penjelasan berikutnya.

## **2.4. Sistem Rekomendasi**

Mengingat proyek ini secara garis besar merupakan proses memilih quotes, maka mekanisme Sistem Rekomendasi juga tentu memiliki peran besar dalam sistem yang kami kembangkan ini. Seperti namanya sistem rekomendasi merupakan algoritma yang dikhususkan untuk memprediksi suatu item, atau dalam kasus tertentu juga suatu label, yang memiliki tingkat kecocokan maksimal terhadap suatu kondisi tertentu, seperti halnya rekomendasi produk atau bahkan film yang mungkin diminati oleh user dengan memperhatikan preferensi khusus user ataupun rekam jejak pilihan user sehingga apa yang dipromosikan pada user dapat lebih terpersonalisasi dengan minat user tersebut.

Pada proyek ini, mekanisme sistem rekomendasi quotes yang dipakai hanya didasarkan pada mekanisme kalkulasi vector sederhana dari query user serta label emosi yang disematkan padanya, dan memilih quote dengan cosine similarity paling mendekati sebagai output quote yang paling direkomendasikan.

## **2.5. React JS**

ReactJS adalah sebuah library JavaScript yang digunakan untuk membangun antarmuka pengguna (UI) dalam aplikasi web. Dikembangkan oleh Facebook, ReactJS memungkinkan developer untuk membuat komponen UI yang dapat digunakan kembali dengan mudah. Salah satu fitur utama ReactJS adalah penggunaan Virtual DOM, yang memungkinkan perubahan pada UI dikomputasi secara efisien, meningkatkan performa aplikasi secara keseluruhan. ReactJS juga mengadopsi paradigma pemrograman yang disebut "reactive programming", di mana perubahan pada data secara otomatis memperbarui tampilan yang sesuai pada UI. Dengan dukungan yang luas dari komunitas developer, serta adopsi oleh perusahaan-perusahaan besar, ReactJS telah menjadi salah satu pilihan utama untuk pengembangan aplikasi web modern.



## 2.6 Flask

Flask adalah sebuah framework web mikro yang ditulis dalam bahasa pemrograman Python. Dikenal karena kesederhanaannya, Flask memberikan kemudahan dalam membangun aplikasi web dengan menyediakan fitur dasar yang diperlukan tanpa memaksakan struktur atau kerangka kerja tertentu. Dengan kemampuan routing yang fleksibel, integrasi yang mudah dengan berbagai ekstensi, dan dukungan untuk pengembangan API RESTful, Flask menjadi pilihan yang populer bagi para pengembang web untuk membangun aplikasi web skala kecil hingga menengah. Di dalam project ini, kami menggunakan Flask sebagai penyambung ReactJS dengan Python. File python yang berbentuk Jupyter Notebook (.ipynb) perlu kami jadikan menjadi .py untuk mempermudah penggunaan Flask untuk aplikasi kami.

## BAB 3. TAHAPAN PELAKSANAAN

### 3.1. EDA dan Preprocessing Data

#### Dataset Link:

- Dataset emosi dari Kaggle:  
<https://www.kaggle.com/datasets/abdallahwagih/emotion-dataset>
- Dataset kumpulan quotes dari Kaggle:  
<https://www.kaggle.com/datasets/akmittal/quotes-dataset/data>

#### Analisis:

Mengingat terdapat dua tipe ML yang dipakai dalam pengembangan proyek ini yakni Semantic Analysis dan Recommendation System, maka analisa terkait EDA dan Preprocess juga akan dibagi dalam dua fase. Berdasarkan hasil pengolahan data menggunakan pemrograman Python Jupyter Notebook didapatkanlah hasil sebagai berikut:

### 3.1.1. Bagian EDA-Preprocessing Dataset Semantic Analysis

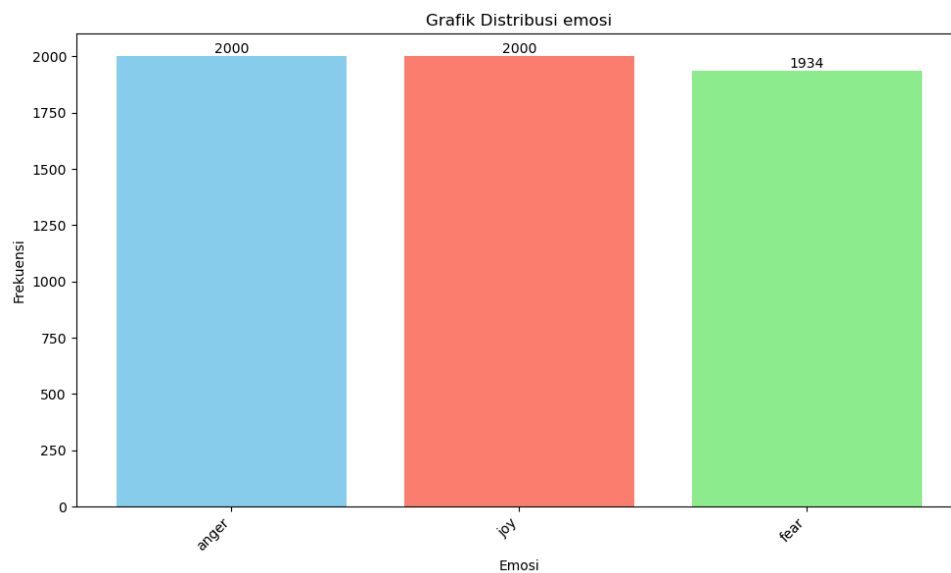
Dataset terdiri dari 2 Kolom yakni kolom '*Comment*' yang berisikan contoh-contoh kalimat pengguna dan kolom '*Emotion*' yang merupakan label emosi dari kalimat tersebut. Dalam kolom Comment terdapat 5937 data dengan 5934 diantaranya merupakan data unik dan tanpa missing value. Sedangkan pada kolom Emotion terdapat pula 5937 data yang terbagi dalam tiga jenis emosi yakni: fear, anger, dan joy. Komposisi data label dalam dataset cukup stabil dimana terdapat 2000 data terlabel sebagai anger, 2000 data dilabel sebagai joy, serta 1937 data dilabel sebagai fear. Dalam dataset tidak ditemukan data NULL.

```
print(dataset.info())
dataset.describe()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5937 entries, 0 to 5936
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Comment 5937 non-null   object
1   Emotion 5937 non-null   object
dtypes: object(2)
memory usage: 92.9+ KB
None
```

|        | Comment                                          | Emotion |
|--------|--------------------------------------------------|---------|
| count  | 5937                                             | 5937    |
| unique | 5934                                             | 3       |
| top    | i feel like a tortured artist when i talk to her | anger   |
| freq   | 2                                                | 2000    |



Unsur ‘ketidak-unikan’ dalam dataset ditemukan pada tiga sel yang duplikat dari kolom Comment. Hal ini tidak dapat dideteksi sebagai suatu tuple yang duplikat dikarenakan Comment yang duplikat tersebut terlabeli Emotion yang berbeda, seperti ditunjukkan pada gambar di bawah.

```
Mode values:
0    i feel like a tortured artist when i talk to her
1    i feel pretty tortured because i work a job an...
2    i resorted to yesterday the post peak day of i...
Name: Comment, dtype: object
Indices of mode values: [2262, 5870, 2877, 4869, 986, 1930]
Emotion values corresponding to the mode values: ['anger', 'fear', 'anger', 'fear', 'anger', 'fear']
```

Untuk mengatasi hal tersebut tuple-tuple yang secara spesifik duplikat pada kolom Comment di-drop salah satunya, dengan menyisakan tuple yang muncul lebih awal. Sampai pada fase ini, dataset baru yang telah diproses inipun disimpan sebagai file csv baru untuk digunakan kembali dalam proses lebih lanjut.

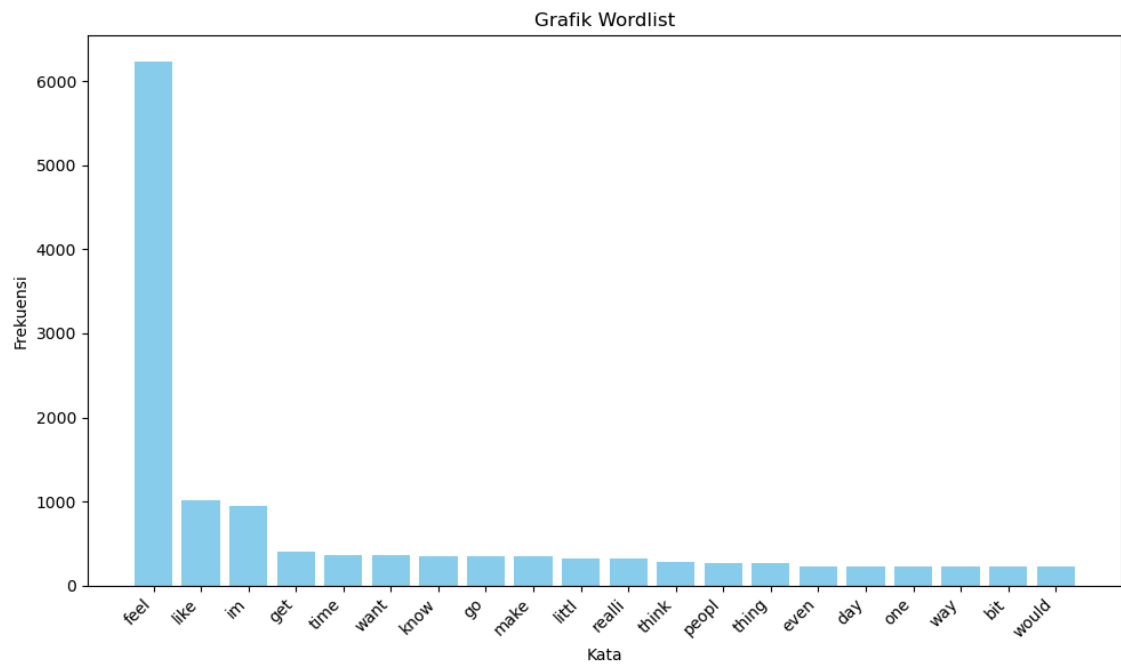
```
datasetNONUnique = dataset
dataset = dataset.drop_duplicates(subset='Comment', keep='first')
✓ 0.0s

dataset.describe()
✓ 0.0s
```

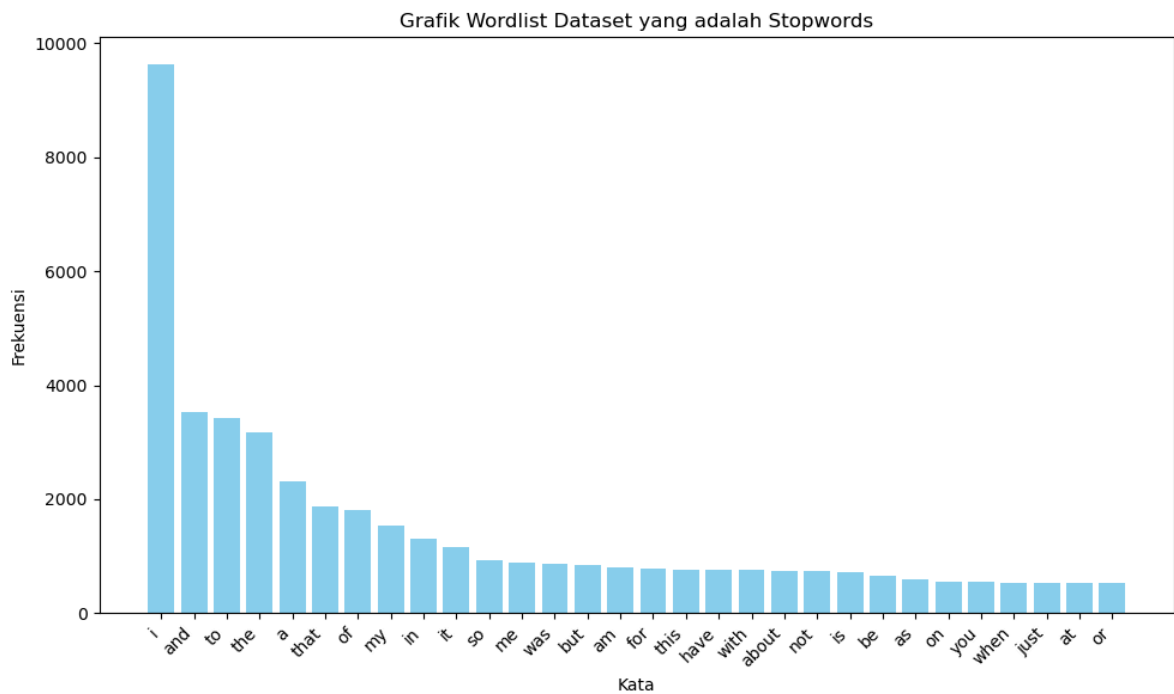
|        | Comment                                           | Emotion |
|--------|---------------------------------------------------|---------|
| count  | 5934                                              | 5934    |
| unique | 5934                                              | 3       |
| top    | i seriously hate one subject to death but now ... | anger   |
| freq   | 1                                                 | 2000    |

Pada bagian preprocessing data selanjutnya, data comment akan pertama-tama dimasukkan dalam satu list besar kata dengan terlebih dahulu di-tokenize dan dibuat lower. Setiap kata dalam list tersebut kemudian melalui proses penghapusan stopwords, punctuation, dan angka, sehingga list hanya berisikan kata-kata yang dapat melatih model dengan efisien. Setiap kata juga di-stemming dan lemmatize sehingga akan terlebih dulu kembali menjadi bentuk akar katanya. Daftar kata dalam list tersebut kemudian dipakai untuk dicocokkan kembali dengan setiap kalimat dalam kolom Comment, yang kemudian dijadikan satu dalam Feature List yang diikuti dengan pemberian label emotionnya. List inilah yang kemudian akan dipakai untuk melatih model.

Setelah melalui fase PreProcessing, berikut adalah komposisi jumlah kata dalam dataset:



Termasuk pula gambaran grafik stopwords yang diperoleh dari dataset sebelum dihilangkan dalam proses remove stopwords:



Dataset yang telah melalui preprocess tahap II (preprocessing setelah data telah disimpan dalam csv kembali sebelumnya) inipun disimpan kembali dalam suatu file baru. Perlu diingat bahwa dataset preprocessed tahap II ini hanya akan diujikan pada model selain BERT. Hal ini berkaitan dengan proses training BERT yang kurang membutuhkan dataset dalam kondisi yang demikian, berhubungan dengan kemampuannya untuk mempelajari pola dalam kondisi kalimat yang utuh seperti akan dijelaskan pada bagian mendatang.

### 3.1.2. Bagian EDA-Preprocessing Dataset Recommendation System

Dalam merekomendasikan quotes melalui mekanisme Recommendation System dipakai suatu dataset yang berbeda dari Kaggle. Perlu dicatat bahwa mengingat label yang dihasilkan melalui model semantic analysis paling optimal akan dipakai untuk mendukung proses sistem rekomendasi, maka sesuai urutan praktiknya, proses ini dilakukan sesudah bagian semantic analysis selesai. Meski begitu dalam laporan ini proses ini dituliskan dalam satu sub-bab dalam EDA dan Preprocessing 3.1 ini.

Dataset quotes terdiri dari 5 kolom tanpa ada nilai null, yakni: Quote, Author, Tags, Popularity, dan Category. Meski begitu, dalam proyek ini, hanya akan digunakan 2 kolom pertama yakni Quote dan Author.

```
#Check if there is NaN Values in Rows
quotes.isnull().sum()
✓ 0.1s
```

|            |       |
|------------|-------|
| Quote      | 0     |
| Author     | 0     |
| Tags       | 0     |
| Popularity | 0     |
| Category   | 0     |
| dtype:     | int64 |

Setelah ditelusuri kondisi dataset dengan baris awal sejumlah 48391 ini memiliki beberapa baris yang duplikat. Lebih dari itu, ditemukan pula beberapa Quote yang duplikat yang diucapkan oleh author yang berbeda.

```
df_selected.describe()
```

✓ 0.1s Python

|        | Quote                                                                                                                 | Author    |
|--------|-----------------------------------------------------------------------------------------------------------------------|-----------|
| count  | 48391                                                                                                                 | 48391     |
| unique | 36937                                                                                                                 | 13839     |
| top    | Your ordinary acts of love and hope point to the extraordinary promise that every human life is of inestimable value. | Beryl Dov |
| freq   | 16                                                                                                                    | 3321      |

Menindaklanjuti hal tersebut, dilakukanlah drop untuk Quote yang duplikat dengan secara khusus menyimpan tuple yang muncul pertama untuk duplikasi dengan Author berbeda seperti dalam tabel di bawah. Total data bersih menjadi 36397 baris.

```
# Drop rows where the "Quote" column is duplicated
df_filtered = df_selected.drop_duplicates(subset='Quote', keep='first')
df_filtered.describe()
```

✓ 0.2s

|        | Quote                                                   | Author    |
|--------|---------------------------------------------------------|-----------|
| count  | 36937                                                   | 36937     |
| unique | 36937                                                   | 13810     |
| top    | Don't cry because it's over, smile because it happened. | Beryl Dov |
| freq   | 1                                                       | 878       |

Proses preprocessing dataset bersih tersebut pun berlanjut dengan menambahkan kolom baru, yakni 'Emotion', dengan mengaplikasikan model semantic analysis optimal tadi pada setiap quote sehingga proses recommendation system nantinya dapat lebih terpersonalisasi pada deteksi emosi dari statement/query yang diinputkan user.

Dataset yang telah terlabeli inipun disimpan kembali dalam bentuk file csv baru sehingga dapat langsung dipanggil pada proses deployment nantinya. Gambar di bawah menjelaskan bukti bagaimana data quotes yang telah dipreprocessing cukup bersih untuk digunakan.

| df_filtered.describe() |                                                         |           |         | Python |
|------------------------|---------------------------------------------------------|-----------|---------|--------|
| ✓                      | 0.0s                                                    |           |         |        |
|                        | Quote                                                   | Author    | emotion |        |
| count                  | 36937                                                   | 36937     | 36937   |        |
| unique                 | 36937                                                   | 13810     | 3       |        |
| top                    | Don't cry because it's over, smile because it happened. | Beryl Dov | joy     |        |
| freq                   | 1                                                       | 878       | 24777   |        |

| df_filtered.head(5) |                                                                                                                                                                                                            |                   |         | Python |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|---------|--------|
| ✓                   | 0.0s                                                                                                                                                                                                       |                   |         |        |
|                     | Quote                                                                                                                                                                                                      | Author            | emotion |        |
| 0                   | Don't cry because it's over, smile because it happened.                                                                                                                                                    | Dr. Seuss         | joy     |        |
| 2                   | I'm selfish, impatient and a little insecure. I make mistakes, I am out of control and at times hard to handle. But if you can't handle me at my worst, then you sure as hell don't deserve me at my best. | Marilyn Monroe    | anger   |        |
| 5                   | Be yourself; everyone else is already taken.                                                                                                                                                               | Oscar Wilde       | anger   |        |
| 6                   | Two things are infinite: the universe and human stupidity; and I'm not sure about the universe.                                                                                                            | Albert Einstein   | joy     |        |
| 9                   | Be who you are and say what you feel, because those who mind don't matter, and those who matter don't mind.                                                                                                | Bernard M. Baruch | joy     |        |

### 3.2. Perancangan Desain Model dan Aplikasi

Dalam mengklasifikasi jenis emosi yang terdapat pada text input, seperti telah disebutkan sebelumnya, kami menggunakan 3 model yang berbeda yakni Naive Bayes Classifier, Logistic Regression serta BERT dari Transformer. Dua model pertama memiliki struktur kode yang cukup mirip karena berasal dari library yang sama yakni nltk.classify. Sedangkan BERT lebih mengaplikasikan Deep Learning dengan pelatihan epoch dengan estimasi 50 atau hingga early stopping aktif.

Di sisi lain, dalam mengembangkan sistem rekomendasi pemilihan quotes, proses didasarkan pada pengukuran kemiripan melalui cosine similarity antar vector input yang dimasukkan user beserta label emosinya dengan setiap vector tuple dalam dataset quotes.

### 3.3. Training-Testing Model

Seperti telah disebutkan sebelumnya, untuk melatih model, Naive Bayes Classifier dan Logistic Regression akan memakai Data Preprocessed tahap I dan II (\*NLTK preprocessing), sedangkan BERT cukup dengan Dataset Preprocessed tahap I. Hal ini ditengarai oleh karakter model BERT yang telah mampu untuk mempelajari data dalam bentuk aslinya, tanpa perlu melalui preprocessing NLP pada umumnya seperti menghilangkan stopwords, stemming, lemmatizing, dan sebagainya. Sebaliknya, penggunaan rangkaian preprocess tambahan itu justru berpotensi memperbesar loss informasi yang disebabkan oleh preprocess tambahan itu sendiri.

Di samping itu, perbedaan ‘asal’ library dari Naive Bayes Classifier (NLTK) dan Logistic Regression (SkLearn) dalam proyek ini juga membuat beberapa modifikasi kecil pada format bentuk dataset, sebelum masuk pada proses training. Naive Bayes memerlukan bentuk data yang lebih ‘terpecah-pecah’, mengingat fungsi utamanya yang ada pada kalkulasi probabilitas kemunculan token.

```
# Load list from file
with open('../Dataset/(PreprocessedII)sentList.json', 'r') as f:
    sentList = json.load(f)

wordlist = []
for sentence in sentList:
    words = sentence.split(' ')
    for word in words:
        wordlist.append(word)

wordlist

✓ 0.0s

['serious',
'hate',
'one',
'subject',
'death',
'feel',
```

Tangkapan layar di atas menunjukkan proses pengolahan data lebih lanjut dalam menuju proses menggunakan Naive Bayes Classifier. Setelah melalui proses tersebut, dibuatlah feature list yang akan menjadi materi training dengan



menghitung kemunculan kata dalam suatu Comment dibandingkan dengan keseluruhan kata yang ada.

```
labeledData = list(zip(commentList, emotionList))

featureList = []
for sentence, label in labeledData:
    checklist = word_tokenize(sentence.lower())
    checklist = preProcess(checklist)

    features = {}
    for word in wordList:
        features[word] = (word in checklist)

    featureList.append((features, label))
```

Setelah itu, feature list tersebut dishuffle dan displit menjadi data training dan testing dengan rasio 8:2. Pembagian tersebut kemudian dipakai untuk melatih model dan dilanjutkan menghitung akurasi serta beberapa informasi metric penting lainnya. Dapat dilihat, model NaiveBayes ini dapat mencapai akurasi sekitar 88%.

```
random.shuffle(featureList)
idxData = int(len(featureList)*0.8)
trainData = featureList[:idxData]
testData = featureList[idxData:]

classifier = NaiveBayesClassifier.train(trainData)
acc = accuracy(classifier, testData)

classifier.show_most_informative_features(n=5)
print(f'Acc: {acc}')
```

✓ 1m 54.1s

Most Informative Features

|                 |              |   |            |
|-----------------|--------------|---|------------|
| scare = True    | fear : joy   | = | 41.1 : 1.0 |
| terrifi = True  | fear : joy   | = | 39.0 : 1.0 |
| unsur = True    | fear : anger | = | 34.7 : 1.0 |
| vulner = True   | fear : joy   | = | 33.6 : 1.0 |
| paranoid = True | fear : anger | = | 32.0 : 1.0 |

Acc: 0.8812131423757371

Berlanjut pada model Machine Learning selanjutnya, sejatinya, proses melatih model Logistic Regression bawaan SkLearn tidak begitu jauh berbeda. Setelah meload preprocessed dataset yang telah disimpan sebelumnya, langkah selanjutnya adalah dengan mengubah fitur dataset yang ada ke dalam bentuk vector, dilanjutkan dengan membagi dataset ke dalam data training dan testing.

```
# Initialize the CountVectorizer
vectorizer = CountVectorizer()

# Transform the text data into feature vectors
X = vectorizer.fit_transform(processed_texts)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, emotionList, test_size=0.2, random_state=42)
```

Proses dilanjutkan langsung dengan menginisialisasi model, melatih model, dan dilanjutkan dengan melakukan testing dan penghitungan akurasi, di mana pada proyek kali ini model Logistic Regression dapat mencapai nilai yang cukup tinggi dengan 92%.

```
# Initialize the Logistic Regression classifier
classifier = LogisticRegression()

# Train the classifier
classifier.fit(X_train, y_train)
```

```
# Predict the labels for the test set
y_pred = classifier.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Print the classification report
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.9191238416175231
              precision    recall  f1-score   support

     anger       0.89       0.93       0.91       401
      fear       0.95       0.89       0.92       401
       joy       0.93       0.94       0.93       385

 accuracy              0.92       1187
  macro avg       0.92       0.92       0.92       1187
 weighted avg       0.92       0.92       0.92       1187
```

Berlanjut pada uji coba model yang terakhir, yakni melatih kembali pre-trained model BERT menggunakan dataset emosi yang ada, di mana proses pengolahan pemrograman model sedikit lebih kompleks dibandingkan model ujicoba sebelumnya. Setelah dataset yang telah di-preprocess dipanggil kembali dan di-*tokenize* menggunakan tokenizer khusus yang juga disediakan oleh dari BERT, dataset akan masuk dalam proses ekstraksi ID input dan attention mask yang akan dipakai dalam proses training. Hal ini dibutuhkan mengingat proses belajar BERT yang didasarkan pada pengenalan teks berbasis ID dalam kamus token BERT dan pemberian bobot belajar tertentu pada bagian-bagian tertentu yang perlu mendapat perhatian.

Perlu diingat bahwa, pengaplikasian mekanisme early stopping secara manual dalam uji coba model BERT ini membutuhkan bagian dataset khusus sebagai validasi. Oleh sebab itu, dataset emosi tadi pertama-tama dipecah menjadi tiga bagian yakni dataset training - validasi - testing. Berikut merupakan gambar tangkapan layar bagaimana dataset dibagi menjadi tiga bagian dan kemudian dikonversi dalam bentuk tensor.

```
# First, split the data into training+validation and test sets
train_val_inputs, test_inputs, train_val_masks, test_masks, train_val_labels, test_labels = train_test_split(
    input_ids, attention_masks, emotion_labels, random_state=42, test_size=0.2
)

# Then, split the training+validation set into separate training and validation sets
train_inputs, val_inputs, train_masks, val_masks, train_labels, val_labels = train_test_split(
    train_val_inputs, train_val_masks, train_val_labels, random_state=42, test_size=0.25
)

# Convert lists to tensors
train_inputs = torch.tensor(train_inputs)
val_inputs = torch.tensor(val_inputs)
train_masks = torch.tensor(train_masks)
val_masks = torch.tensor(val_masks)
train_labels = torch.tensor(train_labels)
val_labels = torch.tensor(val_labels)
test_inputs = torch.tensor(test_inputs)
test_masks = torch.tensor(test_masks)
test_labels = torch.tensor(test_labels)
```

Setelah data dipecah, ditetapkanlah beberapa parameter sebagai berikut:

```
# Fine-tune BERT on your dataset
learning_rate = 2e-5
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)
loss_fn = torch.nn.CrossEntropyLoss()
```

Cross Entropy dipilih sebagai mekanisme kalkulasi loss dalam setiap epoch dengan dilengkapi oleh Adam Optimizer. Pada awalnya, proses training diestimasi hingga 50 epoch, namun pemakaian mekanisme early stopping dengan patience = 3 membuat proses training hanya memerlukan 6 epoch karena tidak adanya lagi penurunan validation loss yang juga dihitung dan diawasi setiap kali model menyelesaikan training 1 epoch.

```
for epoch in range(50): # Train for 50 epochs
    model.train()
    total_loss = 0
    all_preds = []
    all_labels = []
    for batch in tqdm(train_dataloader, desc=f'Epoch {epoch+1}/50'): # Use tqdm for progress bar
        optimizer.zero_grad()
        # Move batch to GPU
        batch = tuple(t.to(device) for t in batch)
        input_ids, attention_mask, labels = batch

        outputs = model(input_ids, attention_mask=attention_mask)
        logits = outputs.logits

        # Cast logits to Float data type
        logits = logits.float()

        # Calculate loss (ensure labels are Long data type)
        loss = F.cross_entropy(logits, labels.long())
        total_loss += loss.item()

        # Calculate accuracy for graphic later
        preds = torch.argmax(logits, dim=1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

        loss.backward()
        optimizer.step()
    avg_loss = total_loss / len(train_dataloader)
    train_losses.append(avg_loss)
    train_accuracy = accuracy_score(all_labels, all_preds)
    train_accuracies.append(train_accuracy)
    print(f'Epoch {epoch+1}/50, Avg. Loss: {avg_loss}, Train Accuracy: {train_accuracy}')
```

```

model.eval()
val_loss = 0
all_val_preds = []
all_val_labels = []
with torch.no_grad():
    for batch in val_dataloader:
        # Move batch to GPU if available
        batch = tuple(t.to(device) for t in batch)
        input_ids, attention_mask, labels = batch

        # Call model.forward on the same device as the inputs
        outputs = model(input_ids, attention_mask=attention_mask)
        logits = outputs.logits

        # Cast logits to Float data type
        logits = logits.float()

        # Calculate validation loss
        loss = F.cross_entropy(logits, labels.long())
        val_loss += loss.item()

    # Calculate accuracy for graphic later
    preds = torch.argmax(logits, dim=1)
    all_val_preds.extend(preds.cpu().numpy())
    all_val_labels.extend(labels.cpu().numpy())
avg_val_loss = val_loss / len(val_dataloader)
val_losses.append(avg_val_loss)
val_accuracy = accuracy_score(all_val_labels, all_val_preds)
val_accuracies.append(val_accuracy)
print(f'Epoch {epoch+1}/50, Avg. Validation Loss: {avg_val_loss}, Validation Accuracy: {val_accuracy}')

```

Berikut adalah progress perkembangan model selama masa training-validasi:

```

Epoch 1/50: 100%|██████████| 112/112 [01:08<00:00, 1.64it/s]
Epoch 1/50, Avg. Loss: 0.7133128521963954, Train Accuracy: 0.6716292134831461
Epoch 1/50, Avg. Validation Loss: 0.2514359219685981, Validation Accuracy: 0.9123841617523167
Epoch 2/50: 100%|██████████| 112/112 [01:07<00:00, 1.66it/s]
Epoch 2/50, Avg. Loss: 0.15232663912632102, Train Accuracy: 0.952808988764045
Epoch 2/50, Avg. Validation Loss: 0.15468391484433883, Validation Accuracy: 0.9486099410278012
Epoch 3/50: 100%|██████████| 112/112 [01:07<00:00, 1.66it/s]
Epoch 3/50, Avg. Loss: 0.05538700775027142, Train Accuracy: 0.9825842696629213
Epoch 3/50, Avg. Validation Loss: 0.1419858609840862, Validation Accuracy: 0.953664700926706
Epoch 4/50: 100%|██████████| 112/112 [01:07<00:00, 1.66it/s]
Epoch 4/50, Avg. Loss: 0.030819230463488827, Train Accuracy: 0.9907303370786517
Epoch 4/50, Avg. Validation Loss: 0.1433504554565604, Validation Accuracy: 0.9502948609941028
Epoch 5/50: 100%|██████████| 112/112 [01:07<00:00, 1.66it/s]
Epoch 5/50, Avg. Loss: 0.03288639176337581, Train Accuracy: 0.9890449438202247
Epoch 5/50, Avg. Validation Loss: 0.15705603908894486, Validation Accuracy: 0.9519797809604044
Epoch 6/50: 100%|██████████| 112/112 [01:07<00:00, 1.66it/s]
Epoch 6/50, Avg. Loss: 0.01909474371184063, Train Accuracy: 0.9938202247191011
Epoch 6/50, Avg. Validation Loss: 0.16489387515589202, Validation Accuracy: 0.9604043807919124
Validation loss has not improved for 3 epochs. Early stopping...

```

Proses berlanjut dengan mentesting model untuk menentukan tingkat performa model yang dilihat dari akurasi. Secara sederhana, proses evaluasi ini berjalan dengan mengambil nilai rata-rata dari seberapa banyak model dapat

secara tepat memprediksi label dari setiap data uji. Seperti pada gambar di bawah dapat dilihat bahwa BERT mendapatkan nilai akurasi yang unggul di angka 95%.

```
model.eval()
total_accurate = 0
test_loss = 0
all_predicted_labels = []
all_true_labels = []

for batch in test_dataloader:
    # Move batch to GPU if available
    batch = tuple(t.to(device) for t in batch)
    input_ids, attention_mask, labels = batch

    # Call prediction
    outputs = model(input_ids.to(device), attention_mask=attention_mask.to(device))
    logits = outputs.logits

    # Cast logits to Long data type
    logits = logits.float()

    # Calculate loss (ensure labels are Long data type)
    loss = F.cross_entropy(logits, labels.long())
    test_loss += loss.item()

    # Calculate performance
    _, predicted_labels = torch.max(logits, dim=1)
    correct_predictions = torch.sum(predicted_labels == labels)
    total_accurate += correct_predictions.item()

    # Append predicted and true labels for confusion matrix calculation
    all_predicted_labels.extend(predicted_labels.cpu().numpy())
    all_true_labels.extend(labels.cpu().numpy())

avg_val_loss = val_loss / len(val_dataloader)
avg_accuracy = total_accurate / len(val_data)
print(f'Validation Loss: {avg_val_loss}, Test Accuracy: {avg_accuracy}')
```

```
Validation Loss: 0.16489387515589202, Test Accuracy: 0.9561920808761584
Confusion Matrix:
[[375  23   4]
 [  2 392   1]
 [  7  15 368]]
```

Model pun disave dalam bentuk .pth untuk kemudian akan digunakan untuk melabeli emosi dataset yang akan dipakai dalam aplikasi quotify

```
# Save the last trained model
checkpoint = {
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'best_val_loss': best_val_loss,
    'early_stop_count': early_stop_count
}

torch.save(checkpoint, 'last_trained_model_checkpoint.pth')
```

Masuk pada pembahasan mengenai model sistem rekomendasi. Mengingat kembali bagian sistem rekomendasi di fase EDA dan Preprocessing seperti telah disebutkan sebelumnya, proses rekomendasi akan menggunakan dataset kumpulan quotes yang telah terlebih dahulu dilabeli prediksi emosinya menggunakan trained model klasifikasi paling optimal yang telah dilatih sebelumnya (*yang adalah BERT model*). Proses rekomendasi quotes didasarkan pada query statement mengenai ungkapan bebas tentang kondisi user di saat itu menginput query. Pemilihan quote berbasis pada randomized pengambilan 5 nilai cosine similarity yang paling mendekati antara input user dengan quote tertentu, dengan ditambahkan dengan bobot similaritas label prediksi emosi tadi.

```
def find_most_similar_quote(input_sentence, quotes_df, input_emotion, emotion_weight=1.1):
    # Combine all quotes into a list
    quotes = quotes_df['Quote'].tolist()
    authors = quotes_df['Author'].tolist()
    emotions = quotes_df['emotion'].tolist()

    # Initialize the vectorizer and fit it on all quotes
    vectorizer = TfidfVectorizer()
    tfidf_matrix = vectorizer.fit_transform(quotes)

    # Vectorize the input sentence
    input_vec = vectorizer.transform([input_sentence])

    # Calculate cosine similarity between the input sentence and all quotes
    cosine_similarities = cosine_similarity(input_vec, tfidf_matrix).flatten()

    # Apply emotion-based weighting
    emotion_weights = np.array([emotion_weight if emotion == input_emotion else 1.0 for emotion in emotions])
    weighted_similarities = cosine_similarities * emotion_weights

    # Get the indices of the top 5 quotes with the highest similarity scores
    top_10_indices = np.argsort(weighted_similarities)[-5:]

    # Randomly select one quote from the top 10
    selected_index = random.choice(top_10_indices)

    # Return the most similar quote and its author
    return quotes[selected_index], authors[selected_index], emotions[selected_index]
```

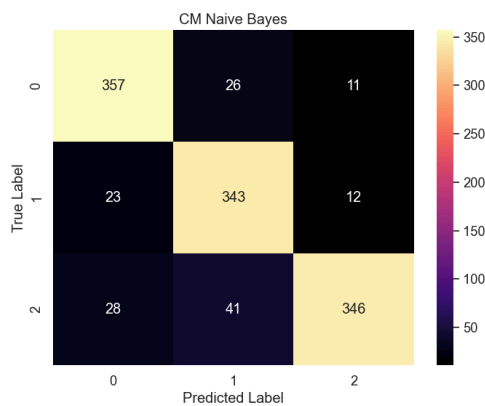
### 3.4. Evaluasi Perangkat dan Uji Coba

Hasil evaluasi menunjukkan model Naive Bayes mampu menggapai akurasi 88%, Logistic Regression di angka 92%, disusul dengan performa BERT yang cukup signifikan sebesar 95%. Hasil tersebut menunjukkan bahwa **BERT melampaui performa model-model lainnya.**

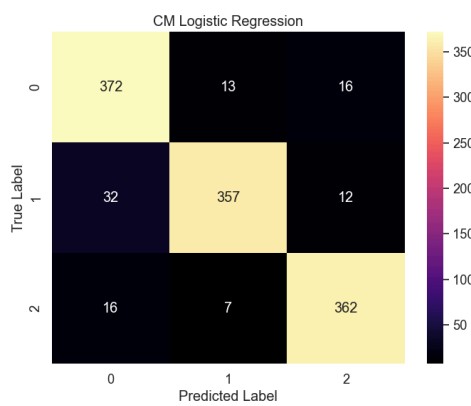
| Model Name             | Accuracy | Precision | Recall | F1 Score |
|------------------------|----------|-----------|--------|----------|
| Naive Bayes Classifier | 88.12%   | 88.46%    | 88.12% | 88.13%   |
| Logistic Regression    | 91.91%   | 92.01%    | 91.91% | 91.91%   |
| BERT                   | 95.61%   | 95.82%    | 95.61% | 95.63%   |

Berikut merupakan informasi tambahan mengenai performa ketiga model tersebut dalam bentuk Confusion matrix:

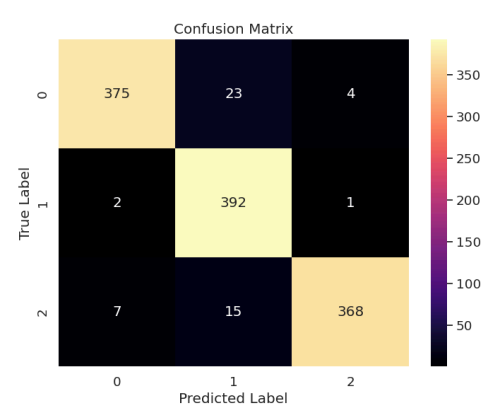
- Confusion Matrix Naive Bayes Classifier
- Confusion Matrix Logistic Regression
- Confusion Matrix BERT



a



b



c



Berikut adalah contoh pengaplikasian prediksi emosi menggunakan BERT:

```
def predict_emotion(text, label_encoder):
    # Tokenize the text using the same tokenizer used during training
    inputs = tokenizer.encode_plus(text, add_special_tokens=True, max_length=128, padding='max_length', truncation=True, return_tensors='pt')
    input_ids = inputs['input_ids'].to(device) # Move input to the same device as the model
    attention_mask = inputs['attention_mask'].to(device) # Move attention mask to the same device as the model

    # Make prediction
    with torch.no_grad():
        model.eval() # Set model to evaluation mode
        outputs = model(input_ids, attention_mask=attention_mask)
    logits = outputs.logits

    # Get predicted label
    predicted_label = torch.argmax(logits, dim=-1).item()

    # Decode the predicted label using label_encoder
    predicted_emotion = label_encoder.inverse_transform([predicted_label])[0]

    return predicted_emotion

# Example usage
text = "I'm happy"
predicted_emotion = predict_emotion(text, label_encoder)
print(f'Predicted Emotion: {predicted_emotion}')
```

Predicted Emotion: joy

Di samping itu, berikut adalah contoh pengaplikasian sistem rekomendasi quotes berbasis pemrograman di Python Jupyter Notebook:

```
input_sentence = "What should I do with my life?"
detected_emotion, emotion_score = predict_emotion(input_sentence, label_encoder)
most_similar_quote, author, emotionQuotes = find_most_similar_quote(input_sentence, quotesDF, detected_emotion)
print('Input sentence: ', input_sentence)
print("Detected emotion:", detected_emotion)
print("Emotion score:", emotion_score)
print("Most similar quote:", most_similar_quote)
print("Author: ", author)
print("Quotes Emotion: ", emotionQuotes)
```

✓ 24s

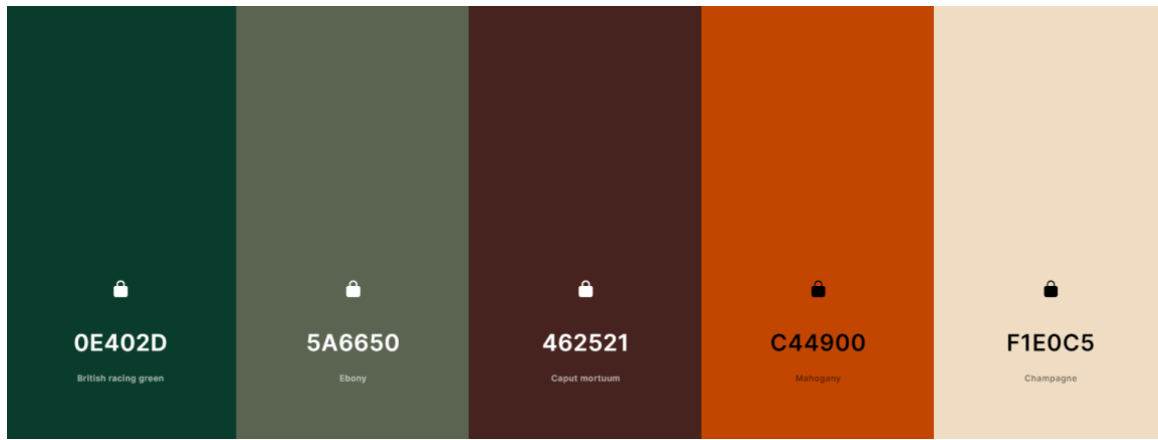
Input sentence: What should I do with my life?  
Detected emotion: anger  
Emotion score: tensor([[ 1.2938, 0.3162, -1.3738]])  
Most similar quote: Before I can tell my life what I want to do with it, I must listen to my life telling me who I am.  
Author: Parker J. Palmer, Let Your Life Speak: Listening for the Voice of Vocation  
Quotes Emotion: joy

## 3.5. Perancangan Aplikasi

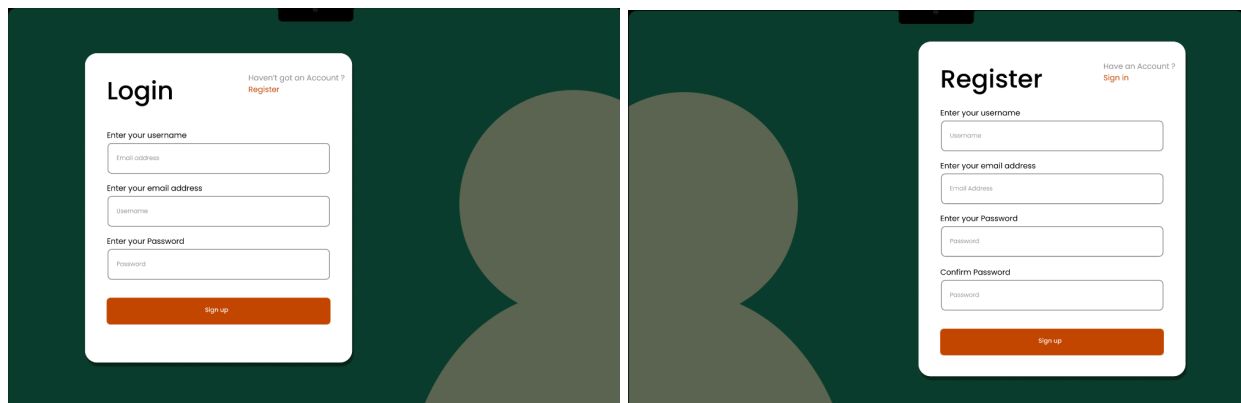
### 3.5.1. Desain Aplikasi

Sebelum membangun aplikasi, tentu saja kita perlu membuat desain aplikasinya terlebih dahulu. Untuk tahap desain, kami menggunakan Figma untuk membuat desainnya.

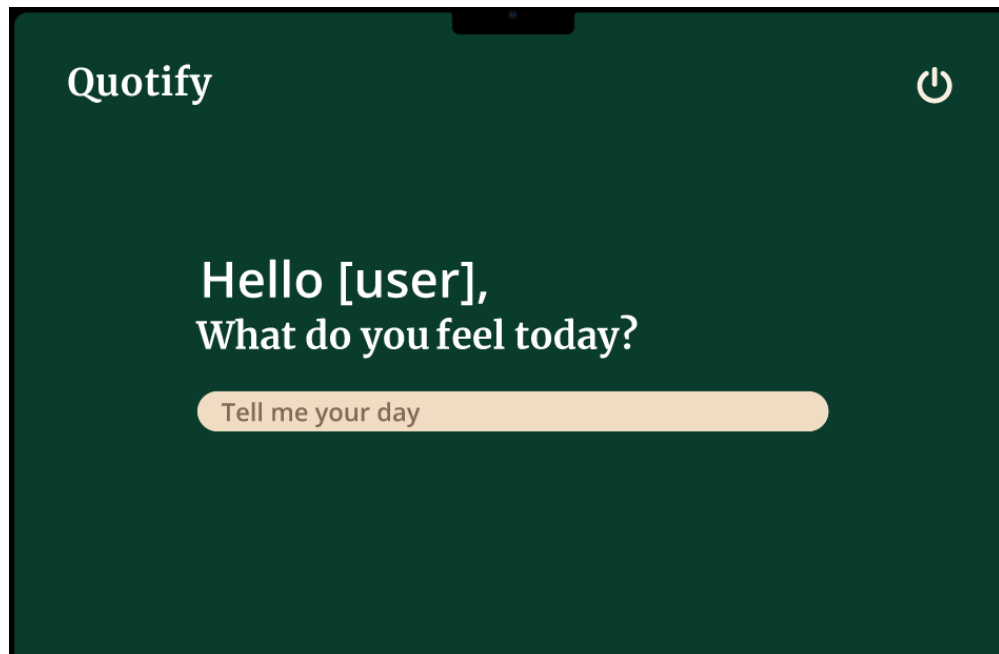
Kami menggunakan color palette hijau yang menggambarkan hijaunya hutan sebagai ekspresi dari mental health:



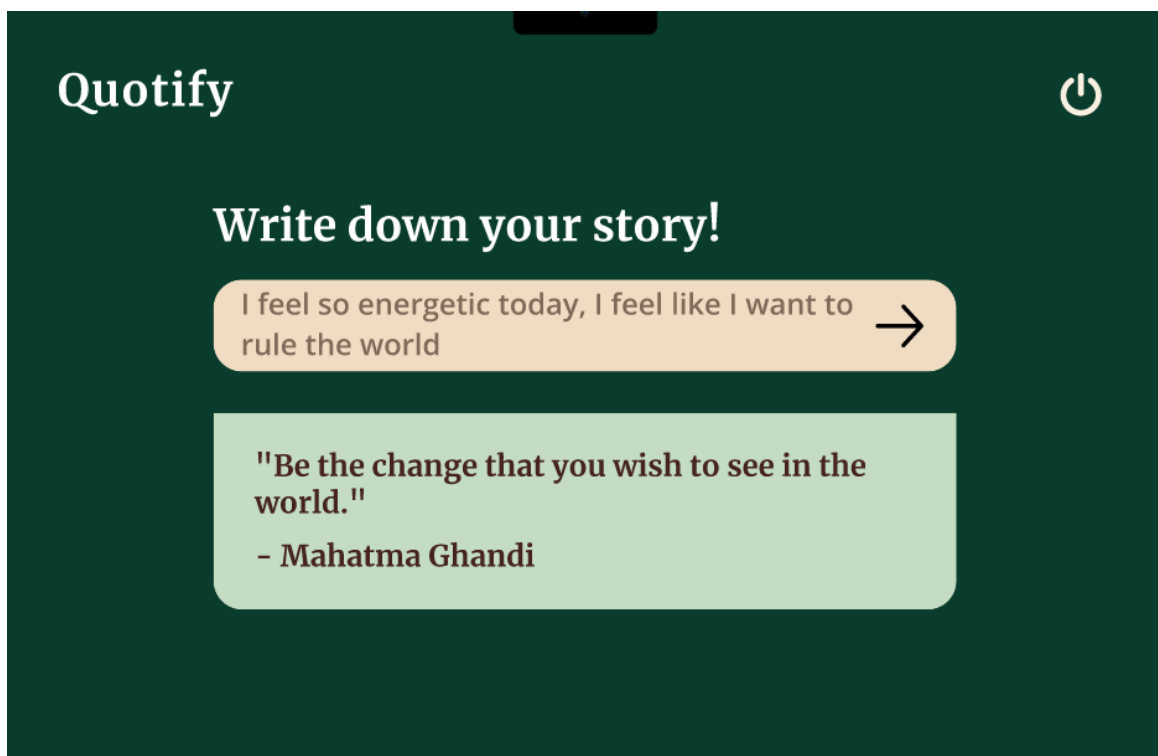
Pada tampilan awal aplikasi, kami mendesain bahwa akan ada login/register untuk mengambil nama dari user.



Setelah user melakukan login, tampilannya akan seperti di bawah ini:

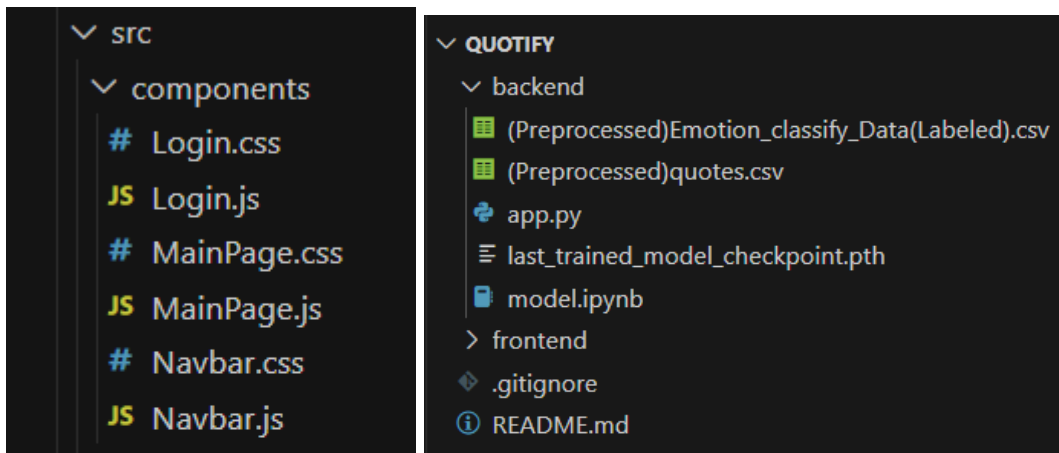


Kemudian user akan diminta untuk memberikan kata-kata mengenai hari dia di hari itu, dan nantinya user akan diberikan kutipan dari tokoh terkenal:



### 3.5.2. Realisasi Aplikasi

Aplikasi kami tentu saja dibagi menjadi 2, yaitu Front End dan Back End. Untuk foldering aplikasi kami, dibagi menjadi 2 folder yaitu folder backend dan folder frontend.



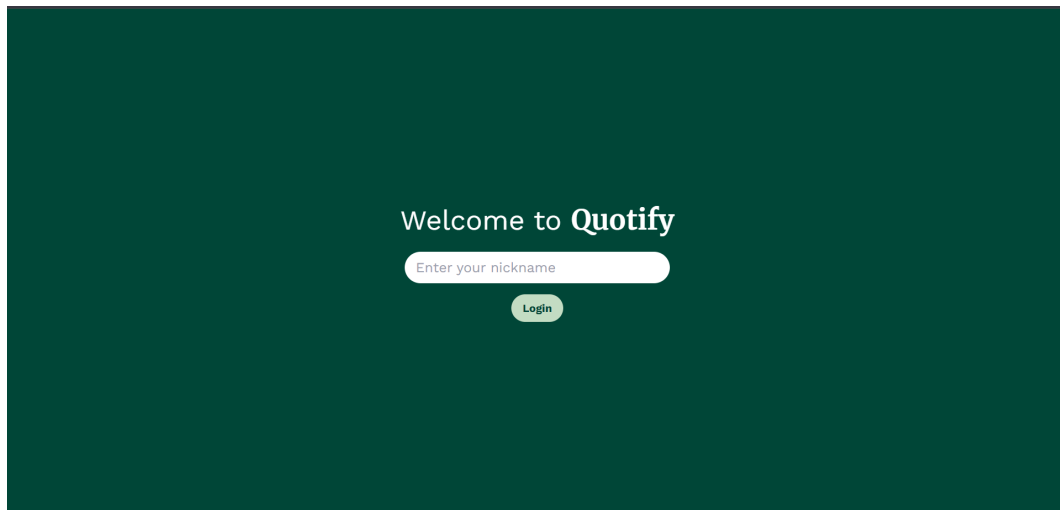
Di folder backend berisikan dataset emosi yang preprocessed dan dataset kutipan. Sebelumnya, untuk modelling kami gunakan file yang memiliki ekstensi ipynb (Jupyter Notebook) namun untuk melakukan integrasi dengan front end, kami gunakan ekstensi py agar lebih mudah penggunaan APInya.

Di folder frontend berisikan folder-folder untuk modul ReactJS dan komponen yang kami gunakan untuk aplikasi kami. Kami membagi frontend kami menjadi 3 bagian saja, Login, MainPage, dan Navbar.

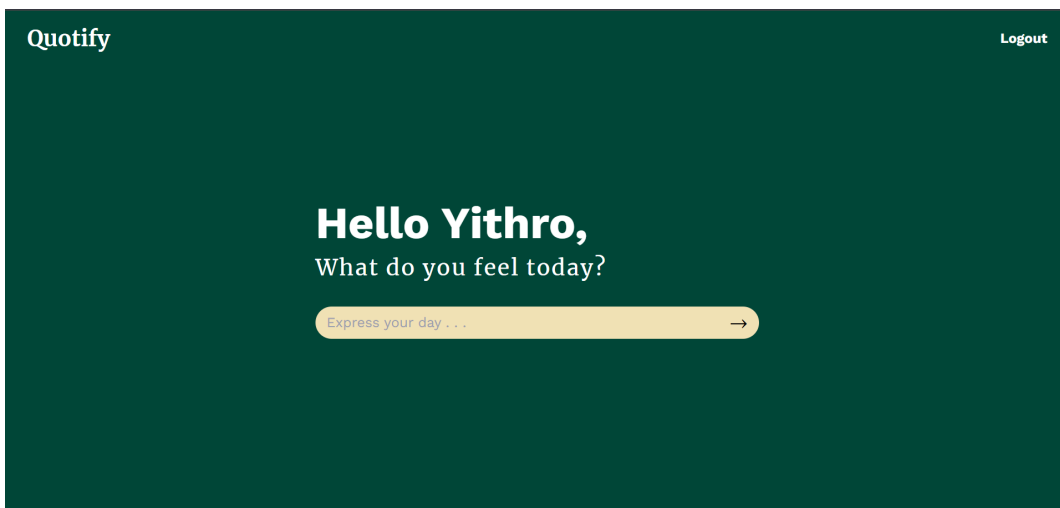
### 3.5.2.1. Front End

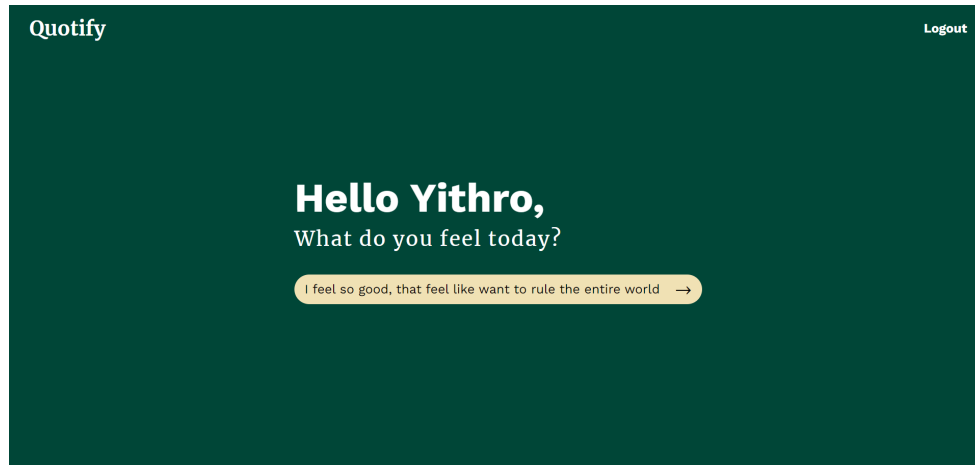
Tech Stack yang kami gunakan untuk membuat FrontEnd aplikasi kami adalah ReactJS. Kami menggunakan ReactJS karena memiliki library yang cukup besar, dan banyak didukung oleh berbagai developer, sehingga kami dapat menggunakan library yang telah mereka kembangkan.

Berikut adalah tampilan dari aplikasi kami.

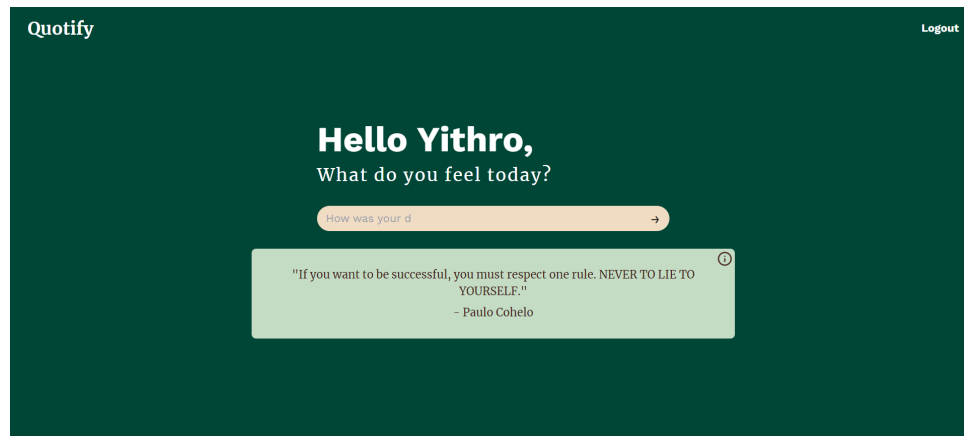


Di login page, kami hanya menggunakan nama untuk login, mengingat keterbatasan waktu kami kurang dapat mengimplementasikan fitur autentikasi, selain itu kami tidak terlalu membutuhkan autentikasi email untuk menggunakan aplikasi kami.

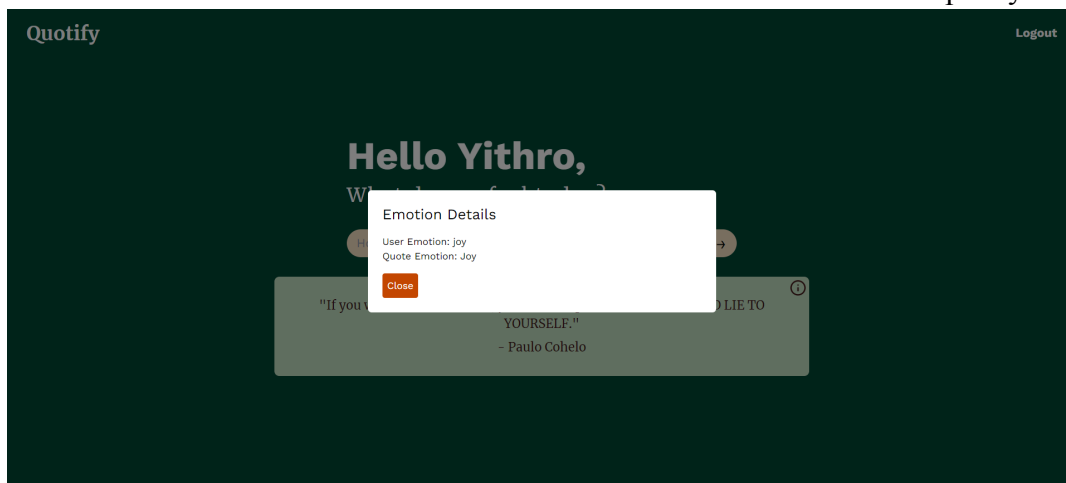




Apabila user melakukan input, dan menekan enter di keyboard ataupun memencet button panah di field input, user akan mendapatkan kutipan dan analisis emosi mereka.



Di bawah ini adalah analisisnya, yang akan keluar apabila user menekan tombol i di sebelah kanan box container dari kutipannya.



### 3.5.2.2. Back End

Tech Stack yang kami gunakan adalah Flask. Karena kami menggunakan Flask, kami harus menggunakan ekstensi .py karena lebih mudah diintegrasikan dibandingkan ekstensi .ipynb.

```
from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app)
```

Untuk menggunakan Flask, hanya diperlukan mengimport librarynya dan mengimport Flask, request, dan jsonify serta CORS. Untuk menginisiasiasinya, kami menggunakan CORS(app) yang dimana app merupakan Flask(\_\_name\_\_).

```
@app.route('/get_quote', methods=['POST'])
def get_quote():
    data = request.get_json()
    input_text = data['inputText']
    detected_emotion, _ = predict_emotion(input_text, label_encoder)
    quote, author, quote_emotion = find_most_similar_quote(input_text, quotesDF, detected_emotion)
    return jsonify({"quote": quote, "author": author, "emotion": detected_emotion, "quote_emotion":
quote_emotion})

if __name__ == '__main__':
    app.run(port=5000, debug=True)
```

Agar model bisa dipakai di front end, kami gunakan routing ke front end menggunakan get\_quote (di bawah ini adalah kodingan di MainPage.js)

```
const handleSubmit = async () => {
    const response = await fetch('http://127.0.0.1:5000/get_quote', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
        },
        body: JSON.stringify({ inputText }),
    });
    const data = await response.json();
    setQuote(data.quote);
    setAuthor(data.author);
    setEmotion(data.emotion);
    setQuoteEmotion(data.quote_emotion);
    setInputText('');
};
```

Untuk mengaktivasi aplikasi kami, pertama harus mengaktifkan backend terlebih dahulu dengan menggunakan perintah “python app.py”

```
PS D:\Kuliah\Semester 4\ML\quotify> cd backend  
PS D:\Kuliah\Semester 4\ML\quotify\backend> python app.py
```

Kemudian baru kita aktivasi frontendnya dengan menggunakan perintah “npm start”

```
PS D:\Kuliah\Semester 4\ML\quotify> cd frontend  
PS D:\Kuliah\Semester 4\ML\quotify\frontend> npm start
```

Berikut merupakan **Link Notion** (berisi lampiran file-file seperti colab, figma, github, dan lainnya): [Link Notion](#)